

AI EDITORIAL FACTORY

DOCUMENTO AGENTICO DI PROGETTO

Architettura, agenti, workflow, dati, sicurezza, pubblicazione e roadmap della redazione multi-agente per manuali tecnici.

Proprietario	Daniel Meloni
Versione documento	1.0
Data	26 agosto 2026
Stato progetto	Fase 7 di 8 completata; Fase 8 progettata

PRINCIPIO GUIDA

Gli agenti propongono, gli esseri umani decidono. Il contenuto originale resta immutabile e ogni pubblicazione richiede approvazione esplicita.

1. Scopo e perimetro

AI Editorial Factory è una piattaforma editoriale multi-agente che trasforma materiali tecnici grezzi — capitoli Markdown, codice, appendici, immagini e fonti — in opere verificate, illustrate e pubblicabili. Il primo pilota è Dataform in Pratica – Volume 1, ma il dominio applicativo è generico e riutilizzabile.

Obiettivi di sistema

- Ridurre il lavoro ripetitivo senza cedere il controllo editoriale all’AI.
- Rendere tracciabile ogni proposta, fonte, revisione, approvazione ed esportazione.
- Separare l’analisi deterministica dall’elaborazione generativa.
- Produrre libri, articoli, lezioni e asset visuali coerenti a partire da una sola base editoriale.
- Estendere il modello dal singolo volume alla gestione completa di collane.

Principi non negoziabili

Principio	Garanzia
Originale immutabile	Ogni intervento crea una nuova versione confrontabile e ripristinabile.
Human-in-the-loop	I workflow si sospendono sui gate; nessuna pubblicazione automatica.
Fonti verificabili	Gli URL proposti vengono aperti e validati prima di essere mostrati.
Tenant isolation	RLS su tutte le tabelle esposte e controlli di appartenenza lato server.
Niente funzioni simulate	Le azioni non implementate sono disabilitate e dichiarate prossime.
Sviluppo economico	Provider mock predefiniti per percorrere il flusso senza crediti AI.

2. Stato del progetto

La piattaforma ha completato sette fasi su otto. Il percorso operativo copre autenticazione, progetto, ingestione, riconoscimento della struttura, audit tecnico, revisione umana, diagrammi, copertina e pubblicazione in Markdown, HTML, PDF ed EPUB. La Fase 8, dedicata alle collane, dispone già di schema dati e modelli di dominio ma non ancora di interfaccia e workflow.

Area	Stato	Nota
Fondazioni e autenticazione	Operativa	Sessioni SSR, PKCE, rotte protette, recupero password.
Database e sicurezza	Operativa	Migration, storage privato, RLS e test PostgreSQL reali.
Agenti e workflow capitolo	Parziale	4 agenti capitolo operativi su 12 previsti; gate umano attivo.
Visual e copertine	Operativa / parziale	Diagrammi e cover programmatici; provider visuali reali da integrare.

Pubblicazione	Operativa	Markdown, HTML, PDF, EPUB, articolo e lezione.
Collane editoriali	Progettata	17 tabelle e regole presenti; UI e 6 agenti ancora assenti.

Baseline dichiarata dal repository: lint e typecheck senza errori, 469 test su 30 file, build con 26 rotte e 22 controlli smoke HTTP.

3. Architettura applicativa

La soluzione adotta un monolite modulare Next.js con App Router. La durabilità dei processi lunghi è delegata a workflow asincroni, evitando microservizi prematuri. Le dipendenze sono unidirezionali: app e componenti dipendono dal dominio; il dominio dipende dagli adapter infrastrutturali Supabase e AI.

Livello	Responsabilità	Regola
app/ e components/	Rotte, Server Component, Server Action, interfaccia utente.	Non contiene logica infrastrutturale sensibile.
lib/ dominio	Validazione, regole editoriali, contratti agenti, workflow, publishing.	Espone funzioni testabili e provider-agnostic.
lib/supabase e lib/ai	Persistenza, sessioni, storage e provider AI.	Implementa interfacce; i segreti restano server-only.

Stack tecnologico

Ambito	Tecnologia
Web	Next.js 16.3, React 19.2, TypeScript 5.9 strict
UI	Tailwind CSS 4.3, primitive accessibili
Dati e identità	Supabase: PostgreSQL, Auth, Storage, SSR
Validazione	Zod 4.4
Workflow	Workflow SDK 4.8
Contenuti	Unified, Remark, Rehype, unpdf, fflate
Output	React PDF, Markdown, HTML, EPUB
Qualità	Vitest, Playwright, ESLint 9, PGLite

Client Supabase

- Browser client: publishable key, nessun segreto nel bundle.

- Server client: sessione da cookie e RLS attiva per componenti, action e route handler.
- Proxy client: rinnova token e cookie e governa i redirect di autenticazione.
- Admin client: service role confinato al server e ammesso solo dopo verifica esplicita dell'organizzazione.

4. Modello agentico

L'agente è un operatore specializzato con un contratto di input/output validato da Zod, configurazione di provider e modello, prompt versionato e tracciamento dell'esecuzione. Il runner non concede potere di pubblicazione: produce risultati strutturati che diventano proposte revisionabili.

Ciclo di esecuzione standard

1. Preparare il contesto minimo necessario: progetto, capitolo, versione, fonti e regole editoriali.
2. Validare l'input con lo schema dell'agente e registrare l'esecuzione.
3. Selezionare il provider tramite registry comune; in sviluppo usare il mock.
4. Eseguire analisi deterministiche prima della generazione AI.
5. Validare l'output strutturato; rifiutare risposte incomplete o fuori contratto.
6. Persistire risultato, motivazioni, riferimenti e metadati di costo/modello.
7. Sospendere il workflow e richiedere revisione o approvazione umana.
8. Applicare soltanto gli elementi approvati creando una nuova versione.

Agenti di capitolo

Agente	Responsabilità	Stato
Technical Accuracy	Verifica codice, configurazioni e affermazioni tecniche.	Operativo
Source & Claims	Rileva claim senza fonte e propone riferimenti verificabili.	Operativo
Structure & Flow	Valuta struttura, ordine e continuità didattica.	Operativo
Editorial Revision	Propone una nuova versione del testo senza alterare l'originale.	Operativo
Style & Tone	Uniformità di voce, leggibilità e convenzioni.	Previsto
Terminology	Glossario, acronimi e coerenza lessicale.	Previsto
Code Review	Correttezza e qualità degli esempi eseguibili.	Previsto
Learning Design	Obiettivi, prerequisiti, esercizi e progressione.	Previsto
Accessibility	Alt text, struttura semantica e fruibilità.	Previsto
Visual Planning	Individua diagrammi e illustrazioni utili.	Previsto
Metadata & SEO	Metadati per web, blog e discovery.	Previsto
Publication Readiness	Controllo finale dei criteri di pubblicazione.	Previsto

Agenti di collana — Fase 8

Agente	Missione
Series Architect	Struttura, posizionamento e roadmap della collana.
Series Curriculum	Progressione didattica, prerequisiti, lacune e duplicazioni.
Series Consistency	Coerenza editoriale, terminologica, visiva e tecnica fra volumi.
Series Visual Director	Identità comune e variazioni controllate per volume.
Cross-Volume Reference	Integrità dei riferimenti fra capitoli e volumi.
Series Publishing	Catalogo, schede volume, scaffale e pacchetto promozionale.

Regola di governance: tutti gli agenti propongono e nessuno applica modifiche definitive senza autorizzazione umana.

5. Workflow editoriale end-to-end

1. Creazione del progetto: autore, titolo, lingua, volume e brief editoriale.
2. Ingestione: caricamento ZIP in storage privato con protezioni contro path traversal e zip bomb.
3. Classificazione: riconoscimento di front matter, parti, capitoli, appendici e asset; ordinamento numerico.
4. Confronto strutturale: indice dichiarato contro cartelle e file realmente presenti.
5. Audit: analisi di SQLX, SQL, JavaScript, claim, fonti, struttura e rischi editoriali.
6. Ricerca fonti: catalogo ufficiale, biblioteca del progetto e ricerca web verificata.
7. Proposta: nuova versione con modifiche granulari, motivazioni e riferimenti.
8. Revisione: diff per righe e parole, accettazione parziale, editing manuale, commenti e ripristino.
9. Visual: diagrammi deterministici e illustrazioni versionate con approvazione.
10. Cover Studio: fronte, dorso e quarta; testi e logo composti programmaticamente.
11. Preflight: controllo stato asset, approvazioni, struttura, fonti e configurazione output.
12. Pubblicazione: generazione di export e accesso tramite URL firmato.

Stati e gate

Ogni esecuzione passa attraverso stati condivisi fra TypeScript e PostgreSQL: in attesa, in esecuzione, sospesa per input umano, completata o fallita. Il gate spezza deliberatamente l'automazione tra proposta e applicazione. Rifiuto e approvazione parziale sono esiti normali, non eccezioni.

6. Fonti, ricerca e verificabilità

Il sistema combina tre canali mantenendo visibile l'origine di ciascun risultato.

Canale	Meccanismo	Vincolo
--------	------------	---------

Catalogo ufficiale	Indice curato e aggiornabile dalle sitemap della documentazione.	Non propone una pagina generica quando non esiste una corrispondenza pertinente.
Biblioteca progetto	Link e PDF caricati dall'utente; PDF indicizzati pagina per pagina.	Ogni risultato conserva origine e pagina.
Web discovery	Provider di ricerca AI, seguito da apertura reale dell'URL.	Le pagine non raggiungibili non vengono mostrate.

- Ogni claim rilevante deve essere collegabile a una fonte o marcato come non verificato.
- Titolo e metadati della fonte derivano dalla pagina letta, non dall'immaginazione del modello.
- Le fonti proposte entrano in una coda di accettazione o scarto umano.
- La bibliografia può essere ricostruita dalle fonti effettivamente approvate.

7. Versionamento e revisione umana

Il contenuto sorgente non viene sovrascritto. Ogni proposta produce una versione figlia collegata al capitolo e all'esecuzione che l'ha generata. La workbench di revisione consente confronto, approvazione per porzioni, modifica manuale, commenti e rollback.

Invarianti

- Una proposta deve essere attribuibile ad agente, provider, modello, prompt e timestamp.
- L'approvazione deve registrare identità, data e perimetro degli elementi accettati.
- Un asset approvato può sostituire il precedente, che passa a superseded senza essere cancellato.
- La pubblicazione usa soltanto versioni e asset nello stato ammesso dal preflight.
- Il ripristino crea un nuovo evento/versione; non cancella la storia.

8. Sistema visuale e copertine

Il progetto separa nettamente ciò che deve essere tecnicamente esatto da ciò che può essere generativo.

Famiglia	Produzione	Proprietà
Diagrammi tecnici	Mermaid o SVG da codice	Deterministici, riproducibili, sempre accompagnati da alt text.
Illustrazioni	Adapter visuale configurabile	Prompt, negative prompt, provider, modello, seed, costo e parentela versionati.
Copertine	Composizione programmatica + eventuale sfondo generato	Titolo, autore, logo, dorso e geometria restano controllabili e verificabili.
Anteprime corsi	SVG 16:9 da codice	Identità coerente, testo esatto e asset autosufficiente.

Regole della copertina

- Larghezza foglio = $2 \times \text{pagina} + \text{dorso} + 2 \times \text{abbondanza}$.
- Il dorso può derivare da mm/pagina, pagine per pollice o valore fisso del fornitore.
- Il valore definitivo si blocca solo con il numero definitivo di pagine.
- Sotto 6 mm il testo verticale del dorso viene omissso.
- Il logo dello strumento viene composto senza ritaglio e non viene ridisegnato dall'AI.
- ISBN validato; il barcode non è composto sulla copertina, ma il generatore EAN-13 resta testato e disponibile.

9. Pubblicazione e derivazioni

Output	Caratteristiche
Markdown	Base portabile e leggibile, coerente con la struttura approvata.
HTML	Conversione tramite Unified/Remark/Rehype e sanitizzazione.
PDF	Composizione con React PDF e stile editoriale controllato.
EPUB	Pacchetto e-book con metadati e navigazione.
Lezione	Derivazione didattica per corso, con anteprima dedicata.
Articolo	Derivazione blog con struttura e metadati dedicati.

Gli export sono persistiti come record di pubblicazione e serviti dal bucket privato tramite URL firmati. La generazione non equivale alla pubblicazione: il sistema richiede che il gate umano e il preflight siano soddisfatti.

10. Modello dati e sicurezza

Lo schema Supabase copre identità e organizzazioni, progetti e fonti, struttura editoriale, agenti e workflow, revisioni, asset visuali, pubblicazioni e collane. Gli enum condividono il vocabolario operativo con il codice TypeScript.

Controlli di sicurezza

- RLS abilitata e forzata sulle tabelle esposte, inclusa la futura area collane.
- Service role esclusivamente server-side e preceduta da verifica esplicita del tenant.
- Bucket privati e URL firmati a breve durata per sorgenti, asset ed export.
- Sanitizzazione HTML e neutralizzazione XML nei testi inseriti in SVG.
- Validazione Zod sugli ingressi e limiti su dimensioni, quantità e struttura degli archivi.
- Protezione dai redirect esterni e rinnovo sicuro della sessione via proxy.
- Audit trail per azioni privilegiate, workflow, approvazioni e pubblicazioni.

Minacce coperte dall'ingestione

Rischio	Difesa
---------	--------

Path traversal	Normalizzazione e guardia dei percorsi prima dell'estrazione.
Zip bomb	Limiti su file, dimensioni compresse/decompresse e volume totale.
File inattesi	Classificazione, whitelist logica e manifest verificato.
Cross-tenant access	RLS e filtri organizzativi lato server.
Asset orfani	Rollback dello storage se il record database non viene creato.

11. Collane editoriali — target Fase 8

La collana è un dominio autonomo. Il legame progetto-volume vive in `series_volumes`, unica fonte di verità, così da evitare duplicazioni e consentire metadati specifici come numero, ISBN, dipendenze, data prevista e deroghe.

Ereditarietà delle regole

Stato	Significato
inherited	Il volume segue la regola di collana e ne riceve gli aggiornamenti.
overridden	Il volume usa una variante locale con motivazione obbligatoria.
locked	La regola è inderogabile; ogni tentativo viene rifiutato.

Le versioni di stile pubblicate sono immutabili. Una modifica genera proposta, analisi d'impatto, anteprima per volume e approvazione. I volumi già pubblicati non vengono aggiornati automaticamente: l'esito è una proposta di nuova edizione.

Fondamenta già presenti

- 17 tabelle di collana con RLS, indici e vincoli.
- Modelli TypeScript e stati del volume.
- Risoluzione `resolveRule()` con rifiuto delle deroghe su regole locked.
- Versionamento di linea editoriale, contenuti condivisi e glossario.
- Modello per riferimenti incrociati, piani di rilascio e issue di coerenza.

Componenti ancora da implementare

- Rotte e interfaccia `/series`.
- Sei agenti di collana e relativi contratti.
- Workflow multi-volume e gate di impatto.
- Vista Scaffale con i dorsi affiancati.
- Export di catalogo, style guide, glossario e pacchetto promozionale.

12. Qualità, test e osservabilità

Livello	Copertura
Unit test	Schemi, agenti, routing AI, ingestione, fonti, cover, publishing, derivazioni e utilità.
Database test	Migration reali e RLS su PostgreSQL via PGlite.
Component test	Navigazione, accessibilità e interazioni principali.
E2E	Autenticazione e percorsi protetti con Playwright.
Smoke test	Controlli HTTP contro la build di produzione.
Static quality	ESLint, TypeScript strict, Prettier e validazione ambiente.

Telemetry minima per esecuzione agente

- ID di esecuzione e correlazione col workflow.
- Agente, versione prompt, provider e modello.
- Input/output validati, durata, tentativi, esito e motivo di errore.
- Token/costo quando disponibili, senza registrare segreti.
- Fonti consultate e decisioni del gate umano.

13. Configurazione e deployment

Il deployment target è Vercel con Supabase gestito. Le variabili pubbliche sono separate dai segreti e la validazione è pigra: la build può completarsi senza credenziali, mentre la prima operazione reale restituisce un errore esplicito.

Gruppo	Variabili principali
Applicazione	NEXT_PUBLIC_APP_URL
Supabase pubblico	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY
Supabase server	SUPABASE_SERVICE_ROLE_KEY
AI testo	AI_TEXT_PROVIDER, AI_TEXT_MODEL, chiave provider
AI immagini	AI_IMAGE_PROVIDER, AI_IMAGE_MODEL
AI ricerca	AI_SEARCH_PROVIDER, AI_SEARCH_MODEL

Runbook essenziale

1. Installare dipendenze e compilare .env.local da .env.example.

2. Eseguire check:env senza stampare segreti.
3. Applicare le migration Supabase e verificare bucket e policy RLS.
4. Eseguire lint, typecheck, test e build.
5. Eseguire lo smoke test sulla build di produzione.
6. Configurare URL applicazione e callback Auth su Supabase e Vercel.
7. Distribuire soltanto dopo approvazione esplicita; mantenere un piano di rollback.

14. Governance operativa degli agenti

Matrice RACI sintetica

Attività	Agente	Editor umano	Sistema
Analizzare	R	A/C	Supporta
Proporre modifica	R	A	Versiona
Accettare/scartare	C	A/R	Registra
Applicare versione	—	A	R, dopo gate
Generare export	R	A	Esegue preflight
Pubblicare	—	A/R	Blocca senza requisiti

Politiche operative

- Minimo privilegio: ogni agente riceve soltanto contesto e strumenti necessari.
- Fail closed: output non valido o fonte non verificabile non avanza il workflow.
- Idempotenza: uno step ripetuto non deve duplicare versioni, asset o pubblicazioni.
- Riproducibilità: prompt, modello, seed e dipendenze devono essere rintracciabili.
- Budget: limiti per progetto, esecuzione e provider; mock come fallback di sviluppo.
- Escalation: conflitti, bassa confidenza e impatto su contenuti pubblicati richiedono revisione umana.

15. Roadmap raccomandata

Priorità	Iniziativa	Criterio di uscita
P0	Completare gli 8 agenti capitolo mancanti	Contratti, runner, UI, test e gate coerenti con i 4 esistenti.
P0	Preflight di pubblicazione unificato	Nessun export finale con fonti, versioni o asset non approvati.
P1	Provider visuali reali	Adapter, cost tracking, retry e moderazione, senza cambiare il dominio.

P1	Fase 8: UI e workflow collane	Creazione collana, ereditarietà, impatto, agenti e vista Scaffale.
P1	Osservabilità agentica	Dashboard di esecuzioni, errori, costi, tempi e approvazioni.
P2	Policy di valutazione continua	Dataset di casi, regression test e scorecard per agente.
P2	Disaster recovery	Backup, ripristino verificato e runbook operativo.

16. Criteri di accettazione globali

- Un progetto completo attraversa ingestione, audit, revisione, visual, cover e pubblicazione senza modificare l'originale.
- Ogni proposta agentica è strutturata, attribuibile, versionata e reversibile.
- Ogni passaggio irreversibile o pubblico richiede approvazione umana esplicita.
- Fonti e URL sono verificabili; l'assenza di evidenza viene dichiarata.
- RLS impedisce accessi fra organizzazioni e nessun segreto raggiunge il client.
- Gli output pubblicati includono soltanto contenuti e asset approvati.
- Diagrammi tecnici sono deterministici; illustrazioni e copertine sono riproducibili e accessibili.
- Lint, typecheck, test database/unit/E2E, build e smoke risultano verdi.
- La collana applica regole ereditate, traccia deroghe e protegge i volumi pubblicati.

17. Glossario

Termine	Definizione
Agente	Componente specializzato che produce un output strutturato e revisionabile.
Workflow	Processo durevole composto da step, stati, retry e gate.
Gate umano	Punto di sospensione che richiede una decisione esplicita.
Claim	Affermazione tecnica verificabile che dovrebbe avere una fonte.
Versione	Snapshot immutabile collegato al contenuto precedente e alla sua origine.
Asset	Diagramma, immagine, copertina o file pubblicabile con stato e metadati.
Derivazione	Contenuto secondario — articolo o lezione — generato dal materiale approvato.
Preflight	Controllo finale dei prerequisiti di pubblicazione.
RLS	Row Level Security: isolamento dei dati applicato dal database.
Superseded	Stato di una versione sostituita ma conservata nello storico.

18. Fonti interne del documento

Questo documento consolida lo stato rilevato nel repository al 26 agosto 2026. Le fonti principali sono:

- README.md
- docs/architecture.md
- docs/agents.md
- docs/workflows.md
- docs/database.md
- docs/security.md
- docs/visual.md
- docs/publishing.md
- docs/deployment.md
- docs/series.md
- src/lib/agents/*
- src/workflows/*
- supabase/migrations/*
- tests/*

Nota: le sezioni di roadmap e governance operativa distinguono esplicitamente le capacità già implementate dalle raccomandazioni per il completamento.